FastAPI Reference

Custom Response Classes - File, HTML, Redirect, Streaming, etc.

There are several custom response classes you can use to create an instance and return them directly from your *path operations*.

Read more about it in the FastAPI docs for Custom Response - HTML, Stream, File, others.

You can import them directly from `fastapi.responses`:

```
from fastapi.responses import (
    FileResponse,
    HTMLResponse,
    JSONResponse,
    ORJSONResponse,
    PlainTextResponse,
    RedirectResponse,
    Response,
    StreamingResponse,
    UJSONResponse,
)
```

FastAPI Responses

There were a couple of custom FastAPI response classes that were intended to optimize JSON performance.

However, they are now deprecated as you will now get better performance by using a Response Model - Return Type.

That way, Pydantic will serialize the data into JSON bytes on the Rust side, which will achieve better performance than these custom JSON responses.

Read more about it in Custom Response - HTML, Stream, File, others - `orjson` or Response Model.

`class` fastapi.responses.UJSONResponse

```
UJSONResponse(
    content,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Bases: `JSONResponse`

JSON response using the ujson library to serialize data to JSON.

Deprecated: `UJSONResponse` is deprecated. FastAPI now serializes data directly to JSON bytes via Pydantic when a return type or response model is set, which is faster and doesn't need a custom response class.

Read more in the FastAPI docs for Custom Response and the FastAPI docs for Response Model.

Note: `ujson` is not included with FastAPI and must be installed separately, e.g. `pip install ujson`.

Ask AI

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    super().__init__(content, status_code, headers, media_type, background)
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = 'application/json'
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `fastapi/responses.py`

```
def render(self, content: Any) -> bytes:
    assert ujson is not None, "ujson must be installed to use UJSONResponse"
    return ujson.dumps(content, ensure_ascii=False).encode("utf-8")
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.ORJSONResponse

```
ORJSONResponse(
    content,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Bases: `JSONResponse`

JSON response using the orjson library to serialize data to JSON.

Deprecated: `ORJSONResponse` is deprecated. FastAPI now serializes data directly to JSON bytes via Pydantic when a return type or response model is set, which is faster and doesn't need a custom response class.

Read more in the FastAPI docs for Custom Response and the FastAPI docs for Response Model.

Note: `orjson` is not included with FastAPI and must be installed separately, e.g. `pip install orjson`.

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    super().__init__(content, status_code, headers, media_type, background)
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = 'application/json'
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `fastapi/responses.py`

```
def render(self, content: Any) -> bytes:
    assert orjson is not None, "orjson must be installed to use ORJSONResponse"
    return orjson.dumps(
        content, option=orjson.OPT_NON_STR_KEYS | orjson.OPT_SERIALIZE_NUMPY
    )
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

Starlette Responses

You can read more about all of them in the FastAPI docs for Custom Response and in the Starlette docs about Responses.

`class` fastapi.responses.FileResponse

```
FileResponse(
    path,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
    filename=None,
    stat_result=None,
    method=None,
    content_disposition_type="attachment",
)
```

Bases: `Response`

Source code in `starlette/responses.py`

```
def __init__(
    self,
    path: str | os.PathLike[str],
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
    filename: str | None = None,
    stat_result: os.stat_result | None = None,
    method: str | None = None,
    content_disposition_type: str = "attachment",
) -> None:
    self.path = path
    self.status_code = status_code
    self.filename = filename
    if method is not None:
        warnings.warn(
            "The 'method' parameter is not used, and it will be removed.",
            DeprecationWarning,
        )
    if media_type is None:
        media_type = guess_type(filename or path)[0] or "text/plain"
    self.media_type = media_type
    self.background = background
    self.init_headers(headers)
    self.headers.setdefault("accept-ranges", "bytes")
    if self.filename is not None:
        content_disposition_filename = quote(self.filename)
        if content_disposition_filename != self.filename:
            content_disposition = f"{content_disposition_type}; filename*=utf-
8''{content_disposition_filename}"
        else:
            content_disposition = f'{content_disposition_type}; filename="{self.filename}"'
        self.headers.setdefault("content-disposition", content_disposition)
    self.stat_result = stat_result
    if stat_result is not None:
        self.set_stat_headers(stat_result)
```

attr chunk_size `class-attribute` `instance-attribute`

```
chunk_size = 64 * 1024
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `instance-attribute`

```
media_type = media_type
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

meth set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```
90  def set_cookie(
91      self,
92      key: str,
93      value: str = "",
94      max_age: int | None = None,
95      expires: datetime | str | int | None = None,
96      path: str | None = "/",
97      domain: str | None = None,
98      secure: bool = False,
99      httponly: bool = False,
100     samesite: Literal["lax", "strict", "none"] | None = "lax",
101     partitioned: bool = False,
102 ) -> None:
103     cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
104     cookie[key] = value
105     if max_age is not None:
106         cookie[key]["max-age"] = max_age
107     if expires is not None:
108         if isinstance(expires, datetime):
109             cookie[key]["expires"] = format_datetime(expires, usegmt=True)
110         else:
111             cookie[key]["expires"] = expires
112     if path is not None:
113         cookie[key]["path"] = path
114     if domain is not None:
115         cookie[key]["domain"] = domain
116     if secure:
117         cookie[key]["secure"] = True
118     if httponly:
119         cookie[key]["httponly"] = True
120     if samesite is not None:
121         assert samesite.lower() in [
122             "strict",
123             "lax",
124             "none",
125         ], "samesite must be either 'strict', 'lax' or 'none'"
126         cookie[key]["samesite"] = samesite
127     if partitioned:
128         if sys.version_info < (3, 14):
129             raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
130 # pragma: no cover
131         cookie[key]["partitioned"] = True  # pragma: no cover
132
133     cookie_val = cookie.output(header="").strip()
        self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
```

```
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.HTMLResponse

```
HTMLResponse(
    content=None,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Bases: `Response`

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any = None,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    self.status_code = status_code
    if media_type is not None:
        self.media_type = media_type
    self.background = background
    self.body = self.render(content)
    self.init_headers(headers)
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = 'text/html'
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.JSONResponse

```
JSONResponse(
    content,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Bases: `Response`

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    super().__init__(content, status_code, headers, media_type, background)
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = 'application/json'
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes:
    return json.dumps(
        content,
        ensure_ascii=False,
        allow_nan=False,
        indent=None,
        separators=(",", ":"),
    ).encode("utf-8")
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.PlainTextResponse

```
PlainTextResponse(
    content=None,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Bases: `Response`

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any = None,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    self.status_code = status_code
    if media_type is not None:
        self.media_type = media_type
    self.background = background
    self.body = self.render(content)
    self.init_headers(headers)
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = 'text/plain'
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.RedirectResponse

```
RedirectResponse(
    url, status_code=307, headers=None, background=None
)
```

Bases: `Response`

Source code in `starlette/responses.py`

```
def __init__(
    self,
    url: str | URL,
    status_code: int = 307,
    headers: Mapping[str, str] | None = None,
    background: BackgroundTask | None = None,
) -> None:
    super().__init__(content=b"", status_code=status_code, headers=headers, background=background)
    self.headers["location"] = quote(str(url), safe=":/%#?=@[]!$&'()*+,;")
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = None
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

meth set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

meth delete_cookie

```
delete_cookie(
    key,
    path="/",
```

```
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.Response

```
Response(
    content=None,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any = None,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    self.status_code = status_code
    if media_type is not None:
        self.media_type = media_type
    self.background = background
    self.body = self.render(content)
    self.init_headers(headers)
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = None
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```

class fastapi.responses.StreamingResponse

```
StreamingResponse(
    content,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Bases: `Response`

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: ContentStream,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    if isinstance(content, AsyncIterable):
        self.body_iterator = content
    else:
        self.body_iterator = iterate_in_threadpool(content)
    self.status_code = status_code
    self.media_type = self.media_type if media_type is None else media_type
    self.background = background
    self.init_headers(headers)
```

attr body_iterator `instance-attribute`

```
body_iterator
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr media_type `instance-attribute`

```
media_type = (
    media_type if media_type is None else media_type
)
```

attr body `instance-attribute`

```
body = render(content)
```

attr background `instance-attribute`

```
background = background
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```python
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```